

JARVIS 2.0: Private, Autonomous Multi-Agent AI Ecosystem

1. Strategic Executive Summary

JARVIS 2.0 represents a modern shift in operational AI infrastructure—transitioning from fragmented cloud dependencies into a fully customized, local enterprise architecture. Developed to bridge high-impact business strategy with elite technical automation, this ecosystem operates entirely on local hardware (Intel i5, 8GB System RAM, NVIDIA RTX 3060 6GB), ensuring absolute data confidentiality, zero data-leakage risks, and zero recurring API token overhead costs.

By shifting workflows into local compute environments, this platform empowers enterprise growth, deep-dive quantitative financial intelligence, automated competitive analysis, and agile content engineering pipelines, completely independent of commercial cloud vulnerabilities.

2. Key Upgrades & Technical Progressions

- **High-Capacity Production Model Stack:** Expanded the inference engine from lightweight testing architectures to a robust, specialized local model cluster featuring **DeepSeek-7B** (for deep code generation and market structural modeling), **Qwen-7B & Mistral AI** (for multi-lingual strategy and document synthesis), alongside efficient nodes like **SmolLM3-3B** and **Gemma 3n-E2B-IT**.
- **Autonomous Operational Workflows:** Integrated the **OpenClaw Agentic Gateway** layer, enabling secure, system-level file management, advanced visual scraping, browser task automation, and continuous lead tracking directly through a custom execution framework.
- **Human-in-the-Loop Governance:** Developed a strict security verification layer via `exec - approvals.json` (Consent Mode). This enterprise guardrail guarantees that the autonomous agent must halt and receive physical operator sign-off (Y/N) before executing any file mutations, network queries, or shell operations.
- **Memory Context Optimization:** Re-engineered the backend initialization to programmatically override default hardware allocation limits, forcing a **32,768 token**

context length directly inside the Ollama execution stack, completely eliminating data bottleneck constraints during deep-dive document processing and market reporting.

- **Omni-Channel User Interface Stack:** Streamlined interactive workflows through Page Assist browser containers for responsive conversational prompting, an executive Streamlit Dashboard for visual asset tracking and market health statistics, and an optimized OpenClaw Terminal UI (TUI) for heavy orchestration tasks.

3. Core Architecture & Microservices

1. Compute & Local Inference Layer

Powered by **Ollama** and standardized via a **LiteLLM Router Proxy**. This configuration creates an OpenAI-compatible API gateway locally on port 4000, allowing seamless model switching depending on the computational complexity of the incoming corporate task.

2. Neural Memory & Caching Network

Utilizes **ChromaDB** as an offline long-term vector database for Retrieval-Augmented Generation (RAG) to ingest market research and company files, eliminating hallucinations. A high-speed, low-latency **Redis Server** functions as the inter-bot message broker to sync real-time variables and automation states.

3. Decoupled Custom Agents

Master Brain (Bot A) exposes an asynchronous FastAPI endpoint on port 6001 to handle tool routing and query entry points. It is supported by a **Quantitative Analysis Engine (Bot B)** that calculates hourly market signals (RSI, MACD, Bollinger Bands) using yfinance and updates active dashboard communication streams.

4. Unified Infrastructure Startup Sequence

To preserve system resources and maximize hardware efficiency, the entire decoupled service array is initialized via a single automated master batch script:

```
@echo off
title JARVIS ENTERPRISE ECOSYSTEM MASTER LAUNCHER

:: Phase 1: Context Memory Horizon Expansion
set OLLAMA_CONTEXT_LENGTH=32768

:: Phase 2: Core Infrastructure Boot
start "Ollama Engine" /min "C:\Users\Admin\AppData\Local\Programs\Ollama\ollama app.exe"
timeout /t 3 /nobreak
start "Redis Cache" /min "C:\Program Files\Redis\redis-server.exe"

:: Phase 3: Cognitive Memory & Routing Base
start "ChromaDB" /min cmd /c "chroma run --host localhost --port 8000"
start "LiteLLM Router" /min cmd /c "litellm --config config/litellm_config.yaml --port 4000"

:: Phase 4: Custom Agent Systems & Autonomous Gateways
start "JARVIS Master Bot" cmd /k "cd /d C:\jarvis\bot_a && python bot_a.py"
start "Trading Insight Bot" cmd /k "cd /d C:\jarvis\trading_bot && python trading_bot.py"
start "OpenClaw Gateway" cmd /k "cd /d C:\openclaw && openclaw gateway run"

:: Phase 5: Executive Interface Delivery
start "Executive Dashboard" /min cmd /c "cd /d C:\jarvis\dashboard && streamlit run dashboard.py"
```

5. Core Port Mapping Reference

Service Layer	Port Mapping	Enterprise Infrastructure Purpose
LiteLLM Router Proxy	4000	API configuration, token mapping, and core load balancing
Bot A (Master Inference)	6001	FastAPI agent core endpoint and intent processing matrix
Bot B (Training Engine)	6002	Asynchronous local pattern analysis and model tuning loops
Trading Bot Engine	6003	Automated background execution and algorithmic portfolio matching
ChromaDB Data Node	8000	Semantic vector index mapping for corporate intelligence retrieval
Redis Memory Cache	6379	Neural communication bus for low-latency multi-process messaging
Executive Dashboard	8501	Streamlit tracking panel for real-time portfolio analytics
Ollama Server Core	11434	Raw physical model weight calculation (DeepSeek, Qwen, Mistral)

6. Strategic Value & Executive Positioning

From an executive alignment perspective, JARVIS 2.0 effectively targets the single primary operational blocker to enterprise AI adoption: **data risk compliance and security vulnerabilities**. By deploying highly optimized 7B architectures directly onto closed, local hardware environments and sealing file and command executions behind an air-gapped security approval wall, this platform proves how a team can execute bleeding-edge autonomous workflows while protecting corporate capital and operational integrity.